

Инструкция и образец

ПП.04.01 «Сопровождение и обслуживание программного обеспечения компьютерных систем»

Сборка приложения C# WPF, затем подключение CSV · 30 индивидуальных вариантов

09.02.07 Информационные системы и программирование · программист

ПМ.04 Сопровождение и обслуживание программного обеспечения компьютерных систем

Учебно-методический комплект

Сервисный журнал учебного центра

Отдельный пример: проектор и ноутбук, одна неполная запись принтера.

От исходного приложения к проверенному обновлению

Материалы для работы

[Скачать исходный проект ServiceDesk \(ZIP\)](#)

Начните с него: соберите и проверьте приложение без CSV.

[Скачать заготовку подключения CSV \(ZIP\)](#)

Откройте после первой проверки, чтобы изучить дефекты импорта и экспорта.

[Скачать исправленный демонстрационный проект \(ZIP\)](#)

Сверяйтесь после самостоятельной диагностики; это решение демонстрационной области.

Задача сопровождения

Оператор сервисного журнала умеет просматривать встроенные заявки, но хочет обмениваться CSV. Сначала докажите исправность окна, затем воспроизведите ошибки обмена. Исправляйте их по отдельности: так повторная проверка показывает причину улучшения. После общей диагностики добавьте выборку для оператора и выполните собственное предметное правило.

Что считать рабочей копией

Распакуйте исходный проект по ссылке под заголовком в папку Source. Для экспериментов создайте её копию Work. Не меняйте Source. CSV-заготовка нужна для сравнения и воспроизведения; исправленный пример открывайте для сверки после собственного исправления. Это три версии одного приложения с одинаковыми именами классов, поэтому их исходники нельзя складывать в один проект.

Среда

В проверенной среде используются Windows, Visual Studio и .NET SDK 10.0.401. В файле ServiceDesk.csproj указан net10.0-windows и UseWPF=true. SDK компилирует проект; Desktop Runtime выполняет WPF-приложение. Если Visual Studio не поддерживает этот SDK или не видит WPF-компоненты, остановите сборку и сообщите преподавателю фактическую ошибку. Не меняйте целевую платформу наугад.

Открытие и контроль исходной версии

Материалы для работы

[Скачать исходный проект ServiceDesk \(ZIP\)](#)

Начните с него: соберите и проверьте приложение без CSV.

Открыть проект

Запустите Visual Studio. На стартовом окне выберите «Открыть проект или решение» (Open a project or solution), найдите распакованный ServiceDesk.csproj. Через «Вид → Обозреватель решений» (View → Solution Explorer) откройте дерево проекта. Разверните MainWindow.xaml: вложенный MainWindow.xaml.cs — код окна. XAML описывает расположение и привязки; обработчик C# отвечает на действие пользователя.

Собрать и запустить

В верхней панели выберите Debug. Нажмите «Сборка → Собрать решение» (Build → Build Solution), затем F5. Сначала исправляют первую ошибку сборки, а не последующие следствия. В окне нажмите «Тестовые данные»: появятся проектор и ноутбук. Нажмите «Просрочено»: одна открытая заявка просрочена, завершённая исключена. Повторная загрузка не должна удваивать список.

Разобрать событие

Откройте MainWindow.xaml во вкладке XAML. У кнопки есть Click="LoadTestData_Click". В коде окна найдите этот метод через Ctrl+F. Он получает List<Ticket>, назначает его ItemsSource и меняет TextBlock StatusText. DataGrid читает значения свойств по Binding. InitializeComponent создаёт элементы, определённые в XAML; без него имена TicketsGrid и StatusText не готовы к использованию.

Контрольная точка до CSV

Журнал заявок — проверка WPF-приложения

Тестовые данные Просрочено

Id	Объект	Ответственный	Срок	Статус
101	Проектор	Ирина	28.09.2026	Открыта
102	Ноутбук	Павел	29.09.2026	Завершена

Тестовых записей: 2

Проверенный запуск исходного приложения. Даты встроенных записей вычисляются относительно даты запуска.

Подключение CSV без потери исходной проверки

Материалы для работы

[Скачать заготовку подключения CSV \(ZIP\)](#)

Откройте после первой проверки, чтобы изучить дефекты импорта и экспорта.

Получить заготовку

Скачайте CSV-заготовку по ссылке под заголовком. Распакуйте её в отдельную папку Draft, откройте её ServiceDesk.csproj в отдельном окне Visual Studio. В Work сохраните исходные App.xaml и ServiceDesk.csproj. Из Draft замените полностью только App.xaml.cs, MainWindow.xaml и MainWindow.xaml.cs. Это согласованное изменение: новые Click, обработчики и чтение CSV появляются вместе. Не добавляйте второй MainWindow.xaml.cs рядом с первым.

Проверить объединённую версию

Соберите Work и запустите. Должны одновременно присутствовать «Тестовые данные», «Открыть CSV», «Экспорт CSV», «Просрочено». Повторите встроенные данные: два объекта и одна просрочка. Существующий сценарий сохраняется, потому что TestData и его обработчик перенесены в CSV-заготовку. Новые функции не должны уничтожать прежнюю контрольную точку.

Происхождение кода

Заготовка является предоставленной версией ServiceDesk с намеренными D1–D3. Дальнейшие фрагменты берутся из исправленного демонстрационного исходника и помещаются только в точно названные участки. Стандартный код Visual Studio — InitializeComponent и сгенерированные файлы obj — вручную не редактируйте.

Входные данные и воспроизведение трёх дефектов

Материалы для работы

[Скачать демонстрационный CSV для проверки импорта](#)

Откройте до запуска, затем загрузите для проверки качества входных строк.

[Скачать демонстрационный CSV без дефектной строки \(demo-valid.csv\)](#)

Используйте для отдельной проверки экспорта и повторного импорта.

[Скачать заготовку подключения CSV \(ZIP\)](#)

Откройте после первой проверки, чтобы изучить дефекты импорта и экспорта.

Прогноз до импорта

Скачайте demo.csv и demo-valid.csv. Откройте их в текстовом редакторе. Заголовок — Id;Item;Owner;Due;Status. Разделитель — точка с запятой, дата — dd.MM.yyyy. В demo.csv две полные записи и неполная физическая строка 4 с Id=103. Номер строки относится к месту в файле, Id — к объекту. Ожидайте две записи и предупреждение, а не потерю корректных заявок.

D1: ошибка всей загрузки

В Work нажмите «Открыть CSV» и выберите demo.csv. Зафиксируйте сообщение ошибки и отсутствие результата загрузки. В MainWindow.xaml.cs найдите CsvStore.Read. Поставьте точку останова щелчком в левом поле перед var cells. Через F10 и «Отладка → Окна → Локальные» (Debug → Windows → Locals) проследите длину массива. У неполной строки только три поля, обращение к индексу 3 уже ошибочно; проверка должна предшествовать любому чтению недостающего поля.

D2 и D3 независимо от D1

Откройте demo-valid.csv. Нажмите «Просрочено»: черновая версия считает две прошлые даты, включая завершённую. Экспортируйте через «Экспорт CSV» в новый файл broken-export.csv и откройте его текстовым редактором: запятая и yyyy-MM-dd расходятся с контрактом. Повторный импорт этого файла не работает. Сохраните наблюдения до правки; после неё результат сравнивают с тем же входом.

D1: результат чтения содержит данные и предупреждения

Материалы для работы

[Скачать демонстрационный CSV для проверки импорта](#)

Откройте до запуска, затем загрузите для проверки качества входных строк.

Согласованная замена

Остановите отладку Shift+F5. В Work/MainWindow.xaml.cs после существующей строки internal record Ticket добавьте следующую одну строку. ReadResult — новый тип результата, который хранит и корректные записи, и причины пропуска. Внутри CsvStore замените целиком метод public static List<Ticket> Read(string path), включая его фигурные скобки, следующим методом Read. Остальные методы пока оставьте прежними.

Код для указанного изменения

```
internal record ReadResult(List<Ticket> Tickets, List<string> Warnings);
```

Код для указанного изменения

```
private const string Header = "Id;Item;Owner;Due;Status";

public static ReadResult Read(string path)
{
    var tickets = new List<Ticket>();
    var warnings = new List<string>();
    var lines = File.ReadAllLines(path, Encoding.UTF8);
    if (lines.Length == 0 || lines[0].TrimStart('\uFEFF') != Header)
        throw new InvalidDataException($"Ожидается заголовок {Header}");
    for (int index = 1; index < lines.Length; index++)
    {
        if (string.IsNullOrWhiteSpace(lines[index])) continue;
        var cells = lines[index].Split(';');
        string? reason = null;
        if (cells.Length != 5) reason = "ожидается пять полей";
        else if (!int.TryParse(cells[0], out _)) reason = "неверный Id";
        else if (string.IsNullOrWhiteSpace(cells[1]) || string.IsNullOrWhiteSpace(cells[2])) reason = "пустое обязательное поле";
    }
}
```

```

else if (!DateTime.TryParseExact(cells[3], "dd.MM.yyyy", CultureInfo.InvariantCulture, DateTimeStyles.None, out _)) reason = "неверная да
та";
else if (cells[4] != "Открыта" && cells[4] != "Завершена") reason = "неверный статус";
if (reason != null) { warnings.Add($"Строка {index + 1}: {reason}"); continue; }
tickets.Add(new Ticket(int.Parse(cells[0], CultureInfo.InvariantCulture), cells[1], cells[2],
    DateTime.ParseExact(cells[3], "dd.MM.yyyy", CultureInfo.InvariantCulture), cells[4]));
}
return new ReadResult(tickets, warnings);
}

```

Смысл чтения

ReadAllLines получает строки в UTF-8. Заголовок проверяется до цикла. Пустые строки не являются заявками; пять полей проверяются до индексов. TryParse возвращает успех вместо исключения; TryParseExact требует однозначную дату. reason содержит первую найденную причину. continue пропускает только текущую строку. index+1 — её физический номер с учётом заголовка. Хорошая строка создаёт Ticket и остаётся доступной.

Завершить зависимое изменение

Внутри MainWindow замените целиком private void LoadCsv(string path) следующим фрагментом. В App.xaml.cs замените весь файл кодом из следующего блока. Это адаптирует запуск и режим самопроверки к ReadResult. Пока эти три связанные замены не закончены, промежуточная сборка может сообщать несовместимые типы; это не контрольная точка.

Код для указанного изменения

```

private void LoadCsv(string path)
{
    try
    {
        var result = CsvStore.Read(path);
        tickets = result.Tickets;
        TicketsGrid.ItemsSource = tickets.ToList();
        StatusText.Text = $"Загружено: {tickets.Count}; пропущено: {result.Warnings.Count}";
        if (result.Warnings.Count > 0)
            MessageBox.Show(string.Join(Environment.NewLine, result.Warnings), "Предупреждения импорта");
    }
    catch (Exception ex) { MessageBox.Show(ex.Message, "Ошибка импорта"); }
}

```

Код для указанного изменения

```
using System.IO;
using System.Windows;

namespace ServiceDeskWpf;

public partial class App : Application
{
    protected override void OnStartup(StartupEventArgs e)
    {
        base.OnStartup(e);
        if (e.Args.Length == 1 && e.Args[0] == "--test-built-in")
        {
            var rows = TestData.Create(DateTime.Today);
            Console.WriteLine($"test-records={rows.Count}; overdue={CsvStore.CountOverdue(rows, DateTime.Today)}");
            Shutdown();
            return;
        }
        if (e.Args.Length == 2 && e.Args[0] == "--self-test")
        {
            var result = CsvStore.Read(e.Args[1]);
            var target = Path.GetTempFileName();
            try
            {
                CsvStore.Write(target, result.Tickets);
                var again = CsvStore.Read(target);
                if (again.Warnings.Count != 0 || !result.Tickets.SequenceEqual(again.Tickets))
                    throw new InvalidOperationException("Повторный импорт не совпал");
                Console.WriteLine($"tickets={result.Tickets.Count}; warnings={result.Warnings.Count}; overdue={CsvStore.CountOverdue(result.Tickets, DateTime.Today)}");
                foreach (var warning in result.Warnings) Console.WriteLine(warning);
            }
            finally { File.Delete(target); }
            Shutdown();
            return;
        }
        new MainWindow(e.Args.Length == 2 && e.Args[0] == "--demo" ? e.Args[1] : null).Show();
    }
}
```



```
}  
}
```

Проверка D1

Теперь соберите Work, запустите F5 и откройте demo.csv. Таблица должна сохранить заявки 101 и 102, сообщение — «Строка 4: ожидается пять полей», состояние — «Загружено: 2; пропущено: 1». Закройте сообщение: строки остаются. Нажмите «Тестовые данные» и проверьте исходный сценарий. Экспорт пока намеренно несовместим — его исправляют далее.

D2: запись соблюдает тот же контракт, что чтение

Материалы для работы

[Скачать демонстрационный CSV без дефектной строки \(demo-valid.csv\)](#)

Используйте для отдельной проверки экспорта и повторного импорта.

[Скачать демонстрационный CSV для проверки импорта](#)

Откройте до запуска, затем загрузите для проверки качества входных строк.

Точное место правки

В Work/MainWindow.xaml.cs, внутри CsvStore, замените целиком public static void Write(string path, IEnumerable<Ticket> rows) следующим методом. Константа Header уже появилась вместе с Read: если её ещё нет, добавьте private const string Header = "Id;Item;Owner;Due;Status"; непосредственно после открывающей скобки CsvStore. Не объявляйте Header дважды.

Код для указанного изменения

```
public static void Write(string path, IEnumerable<Ticket> tickets)
{
    var lines = new List<string> { Header };
    foreach (var t in tickets)
    {
        if (new[] { t.Item, t.Owner, t.Status }.Any(value => value.Contains(';') || value.Contains('\n') || value.Contains('\r')))
            throw new InvalidDataException("В поле нельзя использовать разделитель или перевод строки");
        lines.Add($"{t.Id};{t.Item};{t.Owner};{t.Due.ToString("dd.MM.yyyy", CultureInfo.InvariantCulture)};{t.Status}");
    }
    File.WriteAllLines(path, lines, Encoding.UTF8);
}
```

Зачем две проверки

Сначала формируется весь список строк, затем File.WriteAllLines записывает его в UTF-8. ToString с InvariantCulture сохраняет dd.MM.yyyy независимо от системной культуры. В этом простом контракте нет CSV-кавычек и экранирования: точка с запятой и перевод строки внутри поля отклоняются, чтобы не создать повреждённый файл. Отказ происходит до записи содержимого.



Сравнить значения

Соберите, откройте `demo-valid.csv`, экспортируйте в `roundtrip.csv` и откройте этот результат. В текстовом редакторе сравните заголовки, оба `Id`, `Item`, `Owner`, `Due` и `Status`; одинакового количества недостаточно. Вернитесь к `demo.csv`: предупреждение о строке 4 должно остаться. Экспорт сохраняет загруженные корректные записи и не восстанавливает отвергнутую строку.

D3: просрочка зависит от срока и состояния

Материалы для работы

[Скачать демонстрационный CSV без дефектной строки \(demo-valid.csv\)](#)

Используйте для отдельной проверки экспорта и повторного импорта.

Заменить одно правило

Внутри CsvStore замените целиком метод CountOverdue следующим фрагментом. Не меняйте TestData.CountOverdue: он был правильным и нужен для сравнения. Два правила должны давать одинаковый результат на одинаковых данных.

Код для указанного изменения

```
public static int CountOverdue(IEnumerable<Ticket> tickets, DateTime today) =>
    tickets.Count(t => t.Status != "Завершена" && t.Due.Date < today.Date);
```

Разобрать выражение

Count считает только элементы с истинным условием. Статус завершения исключается до сравнения дат. Due.Date и today.Date сравнивают календарные дни без времени. Срок сегодня ещё не просрочен. Параметр today позволяет проверить правило на заданную дату без изменения часов Windows.

Граничная проверка

Соберите и откройте demo-valid.csv: после «Просрочено» ожидается 1. В копии CSV создайте открытую вчера, завершённую вчера и открытую сегодня — ожидается 1; замените вчера на завтра — ожидается 0. Затем снова загрузите demo.csv и нажмите «Тестовые данные»: два разных источника данных должны оставаться рабочими.

Отбор для оператора без потери исходного списка

Материалы для работы

[Скачать демонстрационный CSV для отбора оператору](#)

Шесть корректных заявок для проверки фильтра, порядка и сохранности полного экспорта.

Демонстрационная задача

В отдельной демонстрационной области оператору нужны незавершённые просроченные заявки по сроку, затем по Id. Полный импорт должен оставаться в памяти и экспортироваться независимо от того, какую выборку сейчас видит оператор. Так пользовательский просмотр не превращается в потерю данных.

Добавить правило и события

В Work/MainWindow.xaml.cs между закрывающей скобкой CsvStore и объявлением MainWindow добавьте целиком класс OperatorView. Внутри MainWindow перед CountOverdue_Click добавьте два обработчика. Затем в MainWindow.xaml замените только контейнер StackPanel с кнопками целиком на следующий WrapPanel. В свойстве Title корневого Window замените прежнюю подпись на «Журнал заявок — WPF версия 1.2», чтобы оператор видел версию обновления. Три фрагмента являются одним изменением; собирайте после всех трёх вставок.

Код для указанного изменения

```
internal static class OperatorView
{
    public static List<Ticket> Select(IEnumerable<Ticket> rows, DateTime today) =>
        rows.Where(t => t.Status == "Открыта" && t.Due.Date < today.Date)
            .OrderBy(t => t.Due).ThenBy(t => t.Id).ToList();
}
```

Код для указанного изменения

```
private void SelectForOperator_Click(object sender, RoutedEventArgs e)
{
    var selected = OperatorView.Select(tickets, DateTime.Today);
}
```

```
TicketsGrid.ItemsSource = selected;
StatusText.Text = $"Для оператора: {selected.Count}; всего: {tickets.Count}";
}

private void ShowAll_Click(object sender, RoutedEventArgs e)
{
    TicketsGrid.ItemsSource = tickets.ToList();
    StatusText.Text = $"Все записи: {tickets.Count}";
}
```

Код для указанного изменения

```
<WrapPanel Margin="0,0,0,10">
    <Button Content="Тестовые данные" Width="140" Height="32" Margin="0,0,8,0" Click="LoadTestData_Click" />
    <Button Content="Открыть CSV" Width="140" Height="32" Margin="0,0,8,0" Click="OpenCsv_Click" />
    <Button Content="Экспорт CSV" Width="140" Height="32" Margin="0,0,8,0" Click="SaveCsv_Click" />
    <Button Content="Просрочено" Width="140" Height="32" Click="CountOverdue_Click" />
    <Button Content="Для оператора" Width="140" Height="32" Margin="8,0,8,0" Click="SelectForOperator_Click" />
    <Button Content="Все записи" Width="140" Height="32" Click="ShowAll_Click" />
</WrapPanel>
```

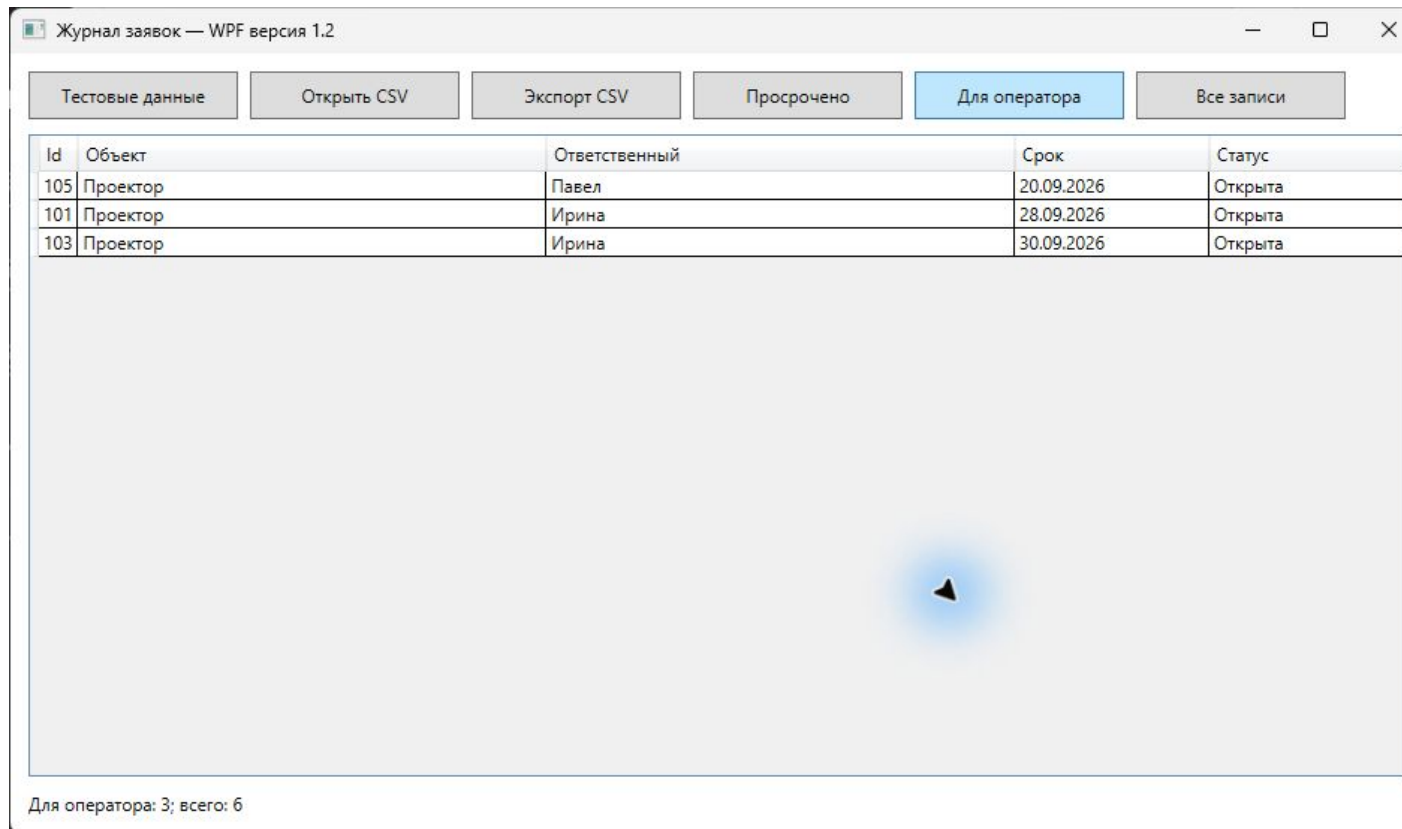
Устройство выборки

Where оставляет нужные записи, OrderBy задаёт главный порядок, ThenBy разрешает равенство срока, ToList создаёт отдельный список для таблицы. tickets не заменяется выборкой. «Все записи» возвращает полный набор. WrapPanel переносит кнопки при недостатке ширины окна. При реализации своего варианта меняется только Select, а загрузка, экспорт и восстановление просмотра сохраняются.

Проверить действие

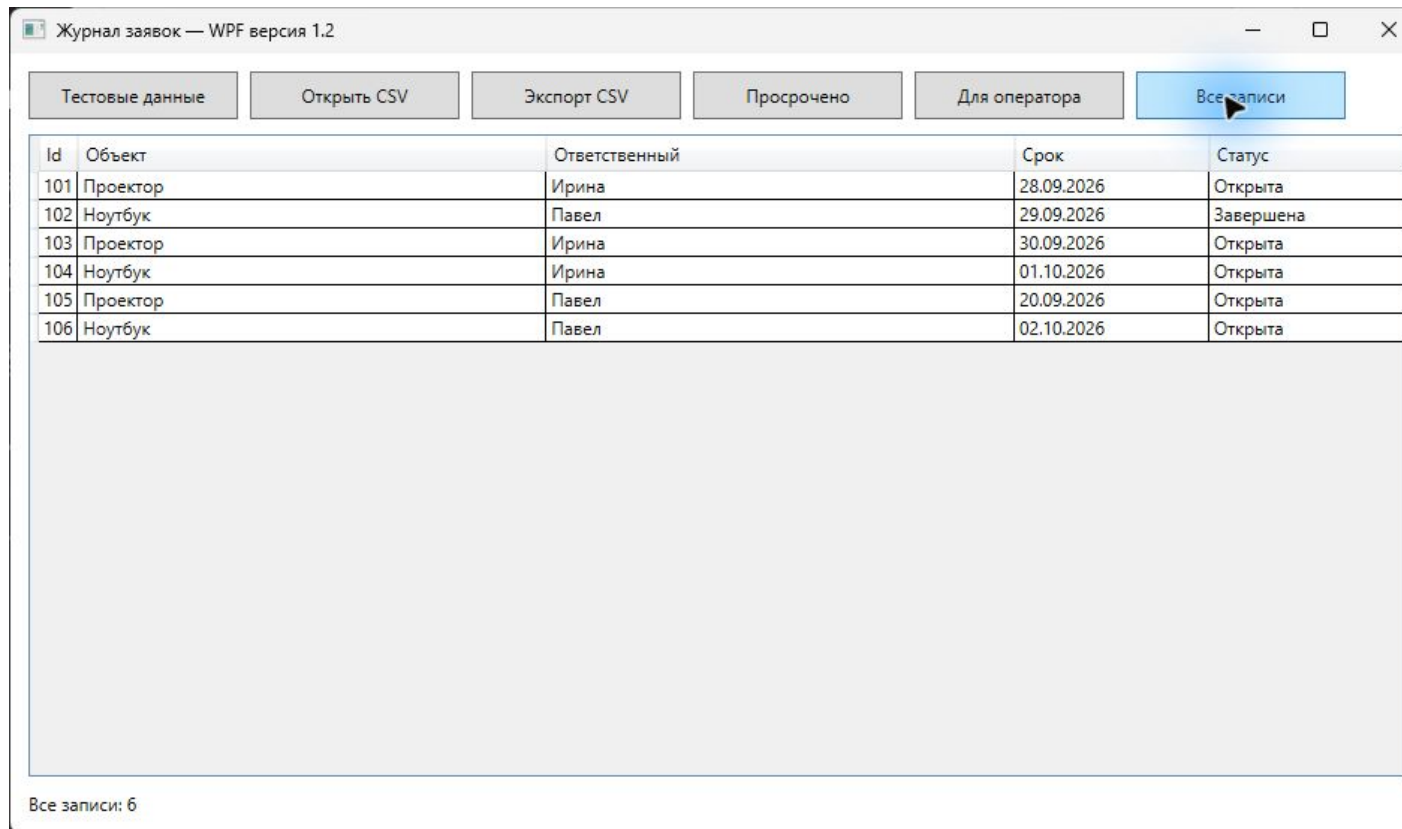
Соберите и загрузите demo-operator.csv по ссылке. Для контрольной даты 01.10.2026 ожидаемые Id выборки — 105, 101, 103. Если системная дата другая, вычислите ожидание заново по той же формуле и запишите дату проверки; часы Windows не меняйте. Нажмите «Для оператора», затем экспортируйте и снова импортируйте: должны сохраниться все шесть записей, а не только выборка. «Все записи» также показывает шесть.

Проверенный отбор



Release-версия на демонстрационном CSV: при дате 01.10.2026 показаны Id 105, 101, 103; полный список содержит шесть записей.

Восстановление просмотра



Кнопка «Все записи» возвращает шесть исходных записей после отбора.

Регрессионная матрица и диагностика ошибок

Набор проверок

На копиях входа проверьте: верный файл; неполную строку после хорошей; неверные Id, дату и статус; пустые Item/Owner; файл только с заголовком; неверный заголовок; отсутствующий файл; строку с лишним разделителем. Для каждого случая укажите вход, действие, ожидаемые записи/сообщение, фактический результат и вывод. Header-only даёт пустой список без предупреждений; неверный заголовок запрещает загрузку; ошибочная отдельная запись не уничтожает хорошие.

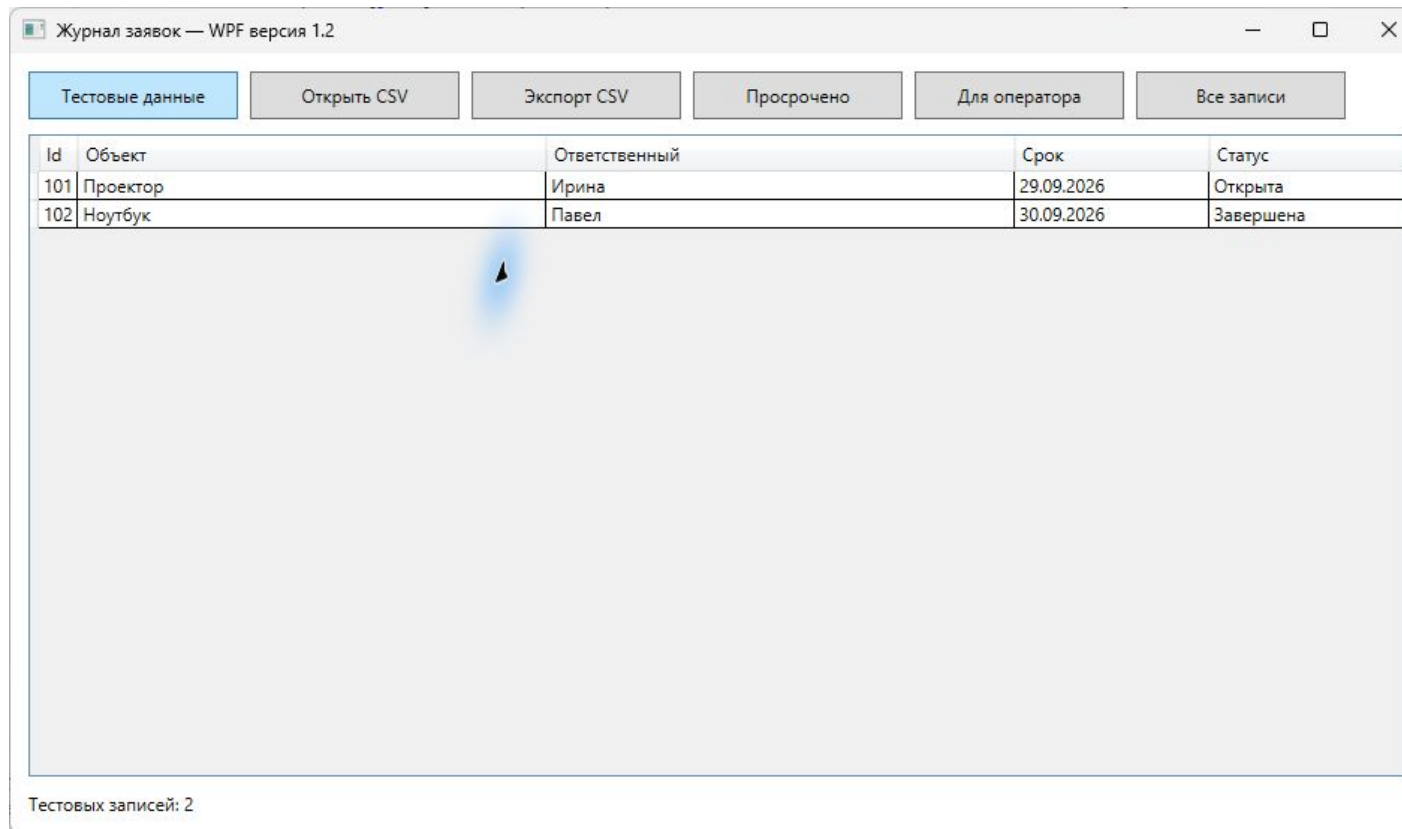
Сохранение после отказа

Загрузите верный CSV, затем попробуйте файл с неверным заголовком. Благодаря назначению tickets только после успешного Read прежние данные остаются в памяти. Экпортируйте их и сравните все поля. Если запись файла запрещена или файл занят, окно сообщает ошибку экспорта; не считайте сообщение «Файл сохранён» доказательством без проверки файла.

Проверка исходной функции

После всех исправлений нажмите «Тестовые данные», «Просрочено», «Для оператора», «Все записи». Ожидаются две встроенные заявки, одна просрочка и восстановление двух строк. После повторного CSV-импорта выборка должна строиться по новому набору. Выполните полный цикл импорта и экспорта ещё раз: проверка отсекает случайные изменения за пределами D1–D3.

Исходная функция сохранена



После изменения приложение по-прежнему загружает две встроенные записи.

Release, резервная копия и передача обновления

Подготовить запускаемую версию

Остановите отладку Shift+F5. В верхней панели Visual Studio выберите Release и «Сборка → Собрать решение». В обозревателе решений щёлкните проект правой кнопкой и выберите «Открыть папку в проводнике» (Open Folder in File Explorer). Откройте bin/Release/net10.0-windows. Скопируйте содержимое этой папки целиком в новую папку Delivery/v1.2; копия одного exe недостаточна. Не берите obj. На целевом Windows должен быть .NET Desktop Runtime 10, соответствующий архитектуре сборки; этот выпуск зависит от установленной среды и не содержит её установщик.

Подготовить резервирование

В отдельной учебной папке Delivery сохраните прежнюю проверенную версию в v1.1, рабочий CSV — в data/work.csv, резервную копию этого CSV — в backup/work-before.csv. Программу и данные храните отдельно. До замены закройте приложение и сравните текст рабочей и резервной копии. Не выполняйте упражнение на реальных данных организации.

Передать и проверить

Запустите exe из v1.2 двойным щелчком, откройте копию data/work.csv. Проверьте предупреждение, две сохранённые записи, счётчик, встроенные данные и предметную выборку. Экспортируйте в data/export-check.csv и повторно импортируйте. Запишите версию, дату, среду, действия и реальные сообщения. Оператор получает понятную инструкцию с названиями кнопок и действиями при ошибке заголовка или чтения.

Проверенный откат и работа по своему варианту

Материалы для работы

[Скачать исправленный демонстрационный проект \(ZIP\)](#)

Сверяйтесь после самостоятельной диагностики; это решение демонстрационной области.

Выполнить откат

Закройте v1.2. Сохраните диагностический экспорт отдельно, замените учебный data/work.csv копией backup/work-before.csv. Запустите прежнюю программу из v1.1 на восстановленной копии. Сравните исходное содержимое CSV и повторите сценарий, который работал до обновления. Намеренные дефекты прежней CSV-версии при этом остаются — откат возвращает прежнее состояние, а не исправляет его. Затем снова закройте приложение и запустите v1.2; резервная копия остаётся нетронутой.

Адаптировать область

Откройте карточку своего варианта: все нужные ссылки повторены там. В проверенной Work-копии поменяйте Window.Title и подписи сущностей, в TestData.Create — значения объектов и ответственных. Сохраните пять полей контракта и обработчик встроенных данных. Сначала повторите исходную проверку, затем загрузите variant-XX.csv: две хорошие записи и предупреждение о строке 4.

Реализовать самостоятельное правило

После D1–D3 реализуйте правило карточки в OperatorView.Select. Для фильтра используйте Where, для порядка OrderBy/ThenBy, для ограничения Take; элементы цепочки возвращают набор записей. Как и в демонстрации, не изменяйте tickets и не превращайте экспорт в экспорт выборки. Отдельный контрольный CSV вашего варианта содержит шесть хороших записей. Сверьте указанный порядок Id на контрольную дату; затем проверьте границы и полный экспорт. Это собственная задача, исправленный пример решает только демонстрационную область.



Описать передачу

Объясните причину каждого исправления, ограничения CSV, правило области, результаты проверки и процедуру отката.

Используйте общий шаблон отчёта с собственной датой, версией и реальными скриншотами. При сдаче проекта приложите ссылку на его репозиторий.

Исходная версия: полный файл ServiceDesk.csproj для сверки

Как использовать листинг

Этот файл предоставлен полностью для чтения и сравнения после соответствующих действий методички. Не добавляйте листинг к уже существующему файлу и не собирайте оборванную часть. Для исправлений используйте точные замены выше.

MainWindow.xaml и его код образуют связанную пару; App запускает именно этот класс окна.

ServiceDesk.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>WinExe</OutputType>
    <TargetFramework>net10.0-windows</TargetFramework>
    <UseWPF>true</UseWPF>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>
</Project>
```

Что сверять

Сверьте имена классов и элементов, Click, формат даты и обработчики. В исправленном окне сохранена загрузка встроенных данных, добавлен CSV и отдельный просмотр для оператора. Если свой вариант отличается предметным правилом, оно должно находиться в OperatorView.Select и проверяться на своих контрольных данных.

Исходная версия: полный файл App.xaml для сверки

Как использовать листинг

Этот файл предоставлен полностью для чтения и сравнения после соответствующих действий методички. Не добавляйте листинг к уже существующему файлу и не собирайте оборванную часть. Для исправлений используйте точные замены выше.

MainWindow.xaml и его код образуют связанную пару; App запускает именно этот класс окна.

App.xaml

```
<Application x:Class="ServiceDeskWpf.App"
             xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">
    <Application.Resources />
</Application>
```

Что сверять

Сверьте имена классов и элементов, Click, формат даты и обработчики. В исправленном окне сохранена загрузка встроенных данных, добавлен CSV и отдельный просмотр для оператора. Если свой вариант отличается предметным правилом, оно должно находиться в OperatorView.Select и проверяться на своих контрольных данных.

Исходная версия: полный файл App.xaml.cs для сверки

Как использовать листинг

Этот файл предоставлен полностью для чтения и сравнения после соответствующих действий методички. Не добавляйте листинг к уже существующему файлу и не собирайте оборванную часть. Для исправлений используйте точные замены выше.

MainWindow.xaml и его код образуют связанную пару; App запускает именно этот класс окна.

App.xaml.cs

```
using System.Windows;

namespace ServiceDeskWpf;

public partial class App : Application
{
    protected override void OnStartup(StartupEventArgs e)
    {
        base.OnStartup(e);
        if (e.Args.Length == 1 && e.Args[0] == "--self-test")
        {
            var rows = TestData.Create(DateTime.Today);
            Console.WriteLine($"test-records={rows.Count}; overdue={TestData.CountOverdue(rows, DateTime.Today)}");
            Shutdown();
            return;
        }
        new MainWindow(e.Args.Length == 1 && e.Args[0] == "--demo").Show();
    }
}
```

Что сверять

Сверьте имена классов и элементов, Click, формат даты и обработчики. В исправленном окне сохранена загрузка встроенных данных, добавлен CSV и отдельный просмотр для оператора. Если свой вариант отличается предметным правилом, оно должно находиться в OperatorView.Select и проверяться на своих контрольных данных.

Исходная версия: полный файл MainWindow.xaml для сверки

Как использовать листинг

Этот файл предоставлен полностью для чтения и сравнения после соответствующих действий методички. Не добавляйте листинг к уже существующему файлу и не собирайте оборванную часть. Для исправлений используйте точные замены выше.

MainWindow.xaml и его код образуют связанную пару; App запускает именно этот класс окна.

MainWindow.xaml

```
<Window x:Class="ServiceDeskWpf.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Журнал заявок – проверка WPF-приложения" Width="960" Height="560">
    <Grid Margin="12">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto" />
            <RowDefinition Height="*" />
            <RowDefinition Height="Auto" />
        </Grid.RowDefinitions>
        <StackPanel Orientation="Horizontal" Margin="0,0,0,10">
            <Button Content="Тестовые данные" Width="150" Height="32" Margin="0,0,8,0" Click="LoadTestData_Click" />
            <Button Content="Просрочено" Width="140" Height="32" Click="CountOverdue_Click" />
        </StackPanel>
        <DataGrid x:Name="TicketsGrid" Grid.Row="1" AutoGenerateColumns="False" IsReadOnly="True">
            <DataGrid.Columns>
                <DataGridTextColumn Header="Id" Binding="{Binding Id}" />
                <DataGridTextColumn Header="Объект" Binding="{Binding Item}" Width="*" />
                <DataGridTextColumn Header="Ответственный" Binding="{Binding Owner}" Width="*" />
                <DataGridTextColumn Header="Срок" Binding="{Binding Due, StringFormat=dd.MM.yyyy}" Width="130" />
                <DataGridTextColumn Header="Статус" Binding="{Binding Status}" Width="130" />
            </DataGrid.Columns>
        </DataGrid>
        <TextBlock x:Name="StatusText" Grid.Row="2" Margin="0,10,0,0" Text="Сначала загрузите встроенные тестовые данные" />
    </Grid>
</Window>
```



Что сверять

Сверьте имена классов и элементов, Click, формат даты и обработчики. В исправленном окне сохранена загрузка встроенных данных, добавлен CSV и отдельный просмотр для оператора. Если свой вариант отличается предметным правилом, оно должно находиться в `OperatorView.Select` и проверяться на своих контрольных данных.

Исходная версия: полный файл MainWindow.xaml.cs для сверки

Как использовать листинг

Этот файл предоставлен полностью для чтения и сравнения после соответствующих действий методички. Не добавляйте листинг к уже существующему файлу и не собирайте оборванную часть. Для исправлений используйте точные замены выше.

MainWindow.xaml и его код образуют связанную пару; App запускает именно этот класс окна.

MainWindow.xaml.cs

```
using System.Windows;

namespace ServiceDeskWpf;

internal record Ticket(int Id, string Item, string Owner, DateTime Due, string Status);

internal static class TestData
{
    public static List<Ticket> Create(DateTime today) =>
    [
        new Ticket(101, "Проектор", "Ирина", today.AddDays(-2), "Открыта"),
        new Ticket(102, "Ноутбук", "Павел", today.AddDays(-1), "Завершена")
    ];

    public static int CountOverdue(IEnumerable<Ticket> tickets, DateTime today) =>
        tickets.Count(t => t.Status != "Завершена" && t.Due.Date < today.Date);
}

public partial class MainWindow : Window
{
    private List<Ticket> tickets = new();

    public MainWindow(bool showTestData = false)
    {
        InitializeComponent();
        if (showTestData) Loaded += (_, _) => LoadTestData();
    }
}
```



```
private void LoadTestData_Click(object sender, RoutedEventArgs e) => LoadTestData();

private void LoadTestData()
{
    tickets = TestData.Create(DateTime.Today);
    TicketsGrid.ItemsSource = tickets.ToList();
    StatusText.Text = $"Тестовых записей: {tickets.Count}";
}

private void CountOverdue_Click(object sender, RoutedEventArgs e) =>
    StatusText.Text = $"Проспечено: {TestData.CountOverdue(tickets, DateTime.Today)}";
}
```

Что сверять

Сверьте имена классов и элементов, Click, формат даты и обработчики. В исправленном окне сохранена загрузка встроенных данных, добавлен CSV и отдельный просмотр для оператора. Если свой вариант отличается предметным правилом, оно должно находиться в `OperatorView.Select` и проверяться на своих контрольных данных.

Исправленная версия: полный файл ServiceDesk-fixed.csproj для сверки

Как использовать листинг

Этот файл предоставлен полностью для чтения и сравнения после соответствующих действий методички. Не добавляйте листинг к уже существующему файлу и не собирайте оборванную часть. Для исправлений используйте точные замены выше.

MainWindow.xaml и его код образуют связанную пару; App запускает именно этот класс окна.

ServiceDesk-fixed.csproj

```
<Project Sdk="Microsoft.NET.Sdk">
  <PropertyGroup>
    <OutputType>WinExe</OutputType>
    <TargetFramework>net10.0-windows</TargetFramework>
    <UseWPF>true</UseWPF>
    <RootNamespace>ServiceDeskWpf</RootNamespace>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>
</Project>
```

Что сверять

Сверьте имена классов и элементов, Click, формат даты и обработчики. В исправленном окне сохранена загрузка встроенных данных, добавлен CSV и отдельный просмотр для оператора. Если свой вариант отличается предметным правилом, оно должно находиться в OperatorView.Select и проверяться на своих контрольных данных.

Исправленная версия: полный файл App.xaml для сверки

Как использовать листинг

Этот файл предоставлен полностью для чтения и сравнения после соответствующих действий методички. Не добавляйте листинг к уже существующему файлу и не собирайте оборванную часть. Для исправлений используйте точные замены выше.

MainWindow.xaml и его код образуют связанную пару; App запускает именно этот класс окна.

App.xaml

```
<Application x:Class="ServiceDeskWpf.App"
             xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
             xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml">
    <Application.Resources />
</Application>
```

Что сверять

Сверьте имена классов и элементов, Click, формат даты и обработчики. В исправленном окне сохранена загрузка встроенных данных, добавлен CSV и отдельный просмотр для оператора. Если свой вариант отличается предметным правилом, оно должно находиться в OperatorView.Select и проверяться на своих контрольных данных.

Исправленная версия: полный файл App.xaml.cs для сверки

Как использовать листинг

Этот файл предоставлен полностью для чтения и сравнения после соответствующих действий методички. Не добавляйте листинг к уже существующему файлу и не собирайте оборванную часть. Для исправлений используйте точные замены выше.

MainWindow.xaml и его код образуют связанную пару; App запускает именно этот класс окна.

App.xaml.cs

```
using System.IO;
using System.Windows;

namespace ServiceDeskWpf;

public partial class App : Application
{
    protected override void OnStartup(StartupEventArgs e)
    {
        base.OnStartup(e);
        if (e.Args.Length == 1 && e.Args[0] == "--test-built-in")
        {
            var rows = TestData.Create(DateTime.Today);
            Console.WriteLine($"test-records={rows.Count}; overdue={CsvStore.CountOverdue(rows, DateTime.Today)}");
            Shutdown();
            return;
        }
        if (e.Args.Length == 2 && e.Args[0] == "--self-test")
        {
            var result = CsvStore.Read(e.Args[1]);
            var target = Path.GetTempFileName();
            try
            {
                CsvStore.Write(target, result.Tickets);
                var again = CsvStore.Read(target);
                if (again.Warnings.Count != 0 || !result.Tickets.SequenceEqual(again.Tickets))
                    throw new InvalidOperationException("Повторный импорт не совпал");
            }
        }
    }
}
```

```
        Console.WriteLine($"tickets={result.Tickets.Count}; warnings={result.Warnings.Count}; overdue={CsvStore.CountOverdue(result.Tickets,
DateTime.Today)}");
        foreach (var warning in result.Warnings) Console.WriteLine(warning);
    }
    finally { File.Delete(target); }
    Shutdown();
    return;
}
new MainWindow(e.Args.Length == 2 && e.Args[0] == "--demo" ? e.Args[1] : null).Show();
}
}
```

Что сверять

Сверьте имена классов и элементов, Click, формат даты и обработчики. В исправленном окне сохранена загрузка встроенных данных, добавлен CSV и отдельный просмотр для оператора. Если свой вариант отличается предметным правилом, оно должно находиться в OperatorView.Select и проверяться на своих контрольных данных.

Исправленная версия: полный файл MainWindow.xaml для сверки

Как использовать листинг

Этот файл предоставлен полностью для чтения и сравнения после соответствующих действий методички. Не добавляйте листинг к уже существующему файлу и не собирайте оборванную часть. Для исправлений используйте точные замены выше.

MainWindow.xaml и его код образуют связанную пару; App запускает именно этот класс окна.

MainWindow.xaml

```
<Window x:Class="ServiceDeskWpf.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        Title="Журнал заявок – WPF версия 1.2" Width="960" Height="560">
    <Grid Margin="12">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto" />
            <RowDefinition Height="*" />
            <RowDefinition Height="Auto" />
        </Grid.RowDefinitions>
        <WrapPanel Margin="0,0,0,10">
            <Button Content="Тестовые данные" Width="140" Height="32" Margin="0,0,8,0" Click="LoadTestData_Click" />
            <Button Content="Открыть CSV" Width="140" Height="32" Margin="0,0,8,0" Click="OpenCsv_Click" />
            <Button Content="Экспорт CSV" Width="140" Height="32" Margin="0,0,8,0" Click="SaveCsv_Click" />
            <Button Content="Просрочено" Width="140" Height="32" Click="CountOverdue_Click" />
            <Button Content="Для оператора" Width="140" Height="32" Margin="8,0,8,0" Click="SelectForOperator_Click" />
            <Button Content="Все записи" Width="140" Height="32" Click="ShowAll_Click" />
        </WrapPanel>
        <DataGrid x:Name="TicketsGrid" Grid.Row="1" AutoGenerateColumns="False" IsReadOnly="True">
            <DataGrid.Columns>
                <DataGridTextColumn Header="Id" Binding="{Binding Id}" />
                <DataGridTextColumn Header="Объект" Binding="{Binding Item}" Width="*" />
                <DataGridTextColumn Header="Ответственный" Binding="{Binding Owner}" Width="*" />
                <DataGridTextColumn Header="Срок" Binding="{Binding Due, StringFormat=dd.MM.yyyy}" Width="130" />
            </DataGrid.Columns>
        </DataGrid>
    </Grid>
</Window>
```

```
<DataGridTextColumn Header="Статус" Binding="{Binding Status}" Width="130" />
</DataGrid.Columns>
</DataGrid>
<TextBlock x:Name="StatusText" Grid.Row="2" Margin="0,10,0,0" Text="Откройте CSV-файл" />
</Grid>
</Window>
```

Что сверять

Сверьте имена классов и элементов, Click, формат даты и обработчики. В исправленном окне сохранена загрузка встроенных данных, добавлен CSV и отдельный просмотр для оператора. Если свой вариант отличается предметным правилом, оно должно находиться в `OperatorView.Select` и проверяться на своих контрольных данных.

Исправленная версия: полный файл MainWindow.xaml.cs для сверки

Как использовать листинг

Этот файл предоставлен полностью для чтения и сравнения после соответствующих действий методички. Не добавляйте листинг к уже существующему файлу и не собирайте оборванную часть. Для исправлений используйте точные замены выше. MainWindow.xaml и его код образуют связанную пару; App запускает именно этот класс окна.

MainWindow.xaml.cs

```
using System.Globalization;
using System.IO;
using System.Text;
using System.Windows;
using Microsoft.Win32;

namespace ServiceDeskWpf;

internal record Ticket(int Id, string Item, string Owner, DateTime Due, string Status);
internal record ReadResult(List<Ticket> Tickets, List<string> Warnings);

internal static class TestData
{
    public static List<Ticket> Create(DateTime today) =>
    [
        new Ticket(101, "Проектор", "Ирина", today.AddDays(-2), "Открыта"),
        new Ticket(102, "Ноутбук", "Павел", today.AddDays(-1), "Завершена")
    ];

    public static int CountOverdue(IEnumerable<Ticket> tickets, DateTime today) =>
        tickets.Count(t => t.Status != "Завершена" && t.Due.Date < today.Date);
}

internal static class CsvStore
```

```

{
    private const string Header = "Id;Item;Owner;Due;Status";

    public static ReadResult Read(string path)
    {
        var tickets = new List<Ticket>();
        var warnings = new List<string>();
        var lines = File.ReadAllLines(path, Encoding.UTF8);
        if (lines.Length == 0 || lines[0].TrimStart('\uFEFF') != Header)
            throw new InvalidDataException($"Ожидается заголовок {Header}");
        for (int index = 1; index < lines.Length; index++)
        {
            if (string.IsNullOrEmpty(lines[index])) continue;
            var cells = lines[index].Split(';');
            string? reason = null;
            if (cells.Length != 5) reason = "ожидается пять полей";
            else if (!int.TryParse(cells[0], out _)) reason = "неверный Id";
            else if (string.IsNullOrEmpty(cells[1]) || string.IsNullOrEmpty(cells[2])) reason = "пустое обязательное поле";
            else if (!DateTime.TryParseExact(cells[3], "dd.MM.yyyy", CultureInfo.InvariantCulture, DateTimeStyles.None, out _)) reason = "неверная да
та";

            else if (cells[4] != "Открыта" && cells[4] != "Завершена") reason = "неверный статус";
            if (reason != null) { warnings.Add($"Строка {index + 1}: {reason}"); continue; }
            tickets.Add(new Ticket(int.Parse(cells[0], CultureInfo.InvariantCulture), cells[1], cells[2],
                DateTime.ParseExact(cells[3], "dd.MM.yyyy", CultureInfo.InvariantCulture), cells[4]));
        }
        return new ReadResult(tickets, warnings);
    }

    public static void Write(string path, IEnumerable<Ticket> tickets)
    {
        var lines = new List<string> { Header };
        foreach (var t in tickets)
        {
            if (new[] { t.Item, t.Owner, t.Status }.Any(value => value.Contains(';') || value.Contains('\n') || value.Contains('\r')))
                throw new InvalidDataException("В поле нельзя использовать разделитель или перевод строки");
            lines.Add($"{t.Id};{t.Item};{t.Owner};{t.Due.ToString("dd.MM.yyyy", CultureInfo.InvariantCulture)};{t.Status}");
        }
        File.WriteAllLines(path, lines, Encoding.UTF8);
    }
}

```

```
public static int CountOverdue(IEnumerable<Ticket> tickets, DateTime today) =>
    tickets.Count(t => t.Status != "Завершена" && t.Due.Date < today.Date);
}

internal static class OperatorView
{
    public static List<Ticket> Select(IEnumerable<Ticket> rows, DateTime today) =>
        rows.Where(t => t.Status == "Открыта" && t.Due.Date < today.Date)
            .OrderBy(t => t.Due).ThenBy(t => t.Id).ToList();
}

public partial class MainWindow : Window
{
    private List<Ticket> tickets = new();

    public MainWindow(string? demoPath = null)
    {
        InitializeComponent();
        if (demoPath != null) Loaded += (_, _) =>
            Dispatcher.BeginInvoke(System.Windows.Threading.DispatcherPriority.ApplicationIdle,
                new Action(() => LoadCsv(demoPath)));
    }

    private void LoadTestData_Click(object sender, RoutedEventArgs e) => LoadTestData();

    private void LoadTestData()
    {
        tickets = TestData.Create(DateTime.Today);
        TicketsGrid.ItemsSource = tickets.ToList();
        StatusText.Text = $"Тестовых записей: {tickets.Count}";
    }

    private void OpenCsv_Click(object sender, RoutedEventArgs e)
    {
        var dialog = new OpenFileDialog { Filter = "CSV (*.csv)|*.csv" };
        if (dialog.ShowDialog() == true) LoadCsv(dialog.FileName);
    }

    private void LoadCsv(string path)
    {

```

```

try
{
    var result = CsvStore.Read(path);
    tickets = result.Tickets;
    TicketsGrid.ItemsSource = tickets.ToList();
    StatusText.Text = $"Загружено: {tickets.Count}; пропущено: {result.Warnings.Count}";
    if (result.Warnings.Count > 0)
        MessageBox.Show(string.Join(Environment.NewLine, result.Warnings), "Предупреждения импорта");
}
catch (Exception ex) { MessageBox.Show(ex.Message, "Ошибка импорта"); }
}

private void SaveCsv_Click(object sender, RoutedEventArgs e)
{
    var dialog = new SaveFileDialog { Filter = "CSV (*.csv)|*.csv", FileName = "export.csv" };
    if (dialog.ShowDialog() != true) return;
    try { CsvStore.Write(dialog.FileName, tickets); StatusText.Text = "Файл сохранён"; }
    catch (Exception ex) { MessageBox.Show(ex.Message, "Ошибка экспорта"); }
}

private void SelectForOperator_Click(object sender, RoutedEventArgs e)
{
    var selected = OperatorView.Select(tickets, DateTime.Today);
    TicketsGrid.ItemsSource = selected;
    StatusText.Text = $"Для оператора: {selected.Count}; всего: {tickets.Count}";
}

private void ShowAll_Click(object sender, RoutedEventArgs e)
{
    TicketsGrid.ItemsSource = tickets.ToList();
    StatusText.Text = $"Все записи: {tickets.Count}";
}

private void CountOverdue_Click(object sender, RoutedEventArgs e) =>
    StatusText.Text = $"Просрочено: {CsvStore.CountOverdue(tickets, DateTime.Today)}";
}

```



Что сверять

Сверьте имена классов и элементов, Click, формат даты и обработчики. В исправленном окне сохранена загрузка встроенных данных, добавлен CSV и отдельный просмотр для оператора. Если свой вариант отличается предметным правилом, оно должно находиться в `OperatorView.Select` и проверяться на своих контрольных данных.